

Code-block density

Code blocks are a common stumbling point: fenced vs indented, info-string tags, content longer than a page, very wide lines, mixed languages back-to-back.

Three languages in a row

```
def hello():
    print("python")
    for i in range(10):
        print(i)
```

```
fn main() {
    println!("rust");
    for i in 0..10 {
        println!("{}", i);
    }
}
```

```
function hello() {
    console.log("javascript");
    for (let i = 0; i < 10; i++) {
        console.log(i);
    }
}
```

Very wide code lines

```
def function_with_a_very_long_name(parameter_one_with_extensive_naming,
parameter_two_also_lengthy, parameter_three_for_good_measure,
parameter_four_just_because):
    return parameter_one_with_extensive_naming +
parameter_two_also_lengthy + parameter_three_for_good_measure +
parameter_four_just_because
```

The line above is about 200 characters. It will not wrap (code blocks render with their literal indentation and no wrapping); it should extend past the right margin if necessary, with no visual corruption.

Code without a language tag

This is a code block with no language hint. The renderer should still tint it light grey and treat the content as verbatim, no inline parsing.

```
    Indented within the code block – preserved.
```

```
    Tab character at start – also preserved.
```

```
A line with markdown syntax that should NOT be parsed.  
A [link](https://example.com) in a code block: not a link.
```

Indented code blocks (4 spaces)

A paragraph immediately before an indented code block.

```
indented_code = "should be a code block"  
next_line = "still in the same block"  
  
blank_line_continues_the_block = True
```

Back to a paragraph.

Code spans of varying lengths

Inline `single_word` and `compound_phrase_with_underscores` and `function_call(arg_one, arg_two, arg_three, arg_four)` and `a_very_long_inline_code_span_that_might_exceed_the_natural_line_width_and_force_a_wrap_at_the_space_boundary_in_the_surrounding_text`.

Code containing backticks

The literal triple-backtick ````` should be expressible in inline code. So should a double backtick ```` (two literal backticks).

A code span with a single backtick ``` (one literal backtick).

Code block containing what looks like a fence

```
This is a code block. The line below LOOKS like a fence but is content.  
```more  
because this line has trailing characters after the three backticks.
But what about a literal triple backtick on its own line?
```

```
That should close the outer block. The line after is plain text.
```

## Mixing inline code and bold/italic

In a sentence: **`**`bold_code`**`** and *`*`italic_code`*`* and  
***`***`bold_italic_code`***`***.

In a list:

- ``simple``
- **`**`bold`**`**
- *`*`italic`*`*
- ***`***`bold_italic`***`***